Lesson 04 - Communicating with the Arduino Nano 33 IOT over UDP – Part 1

In this lesson the student will build an address label maker graphical Qt application that transmits the address label as a JSON payload over UDP to the Arduino Nano 33 IOT which then prints out the label to the serial port and sends back an acknowledgement.

Javascript Object Notation (JSON)

JavaScript Object Notation (JSON) is a standard text-based format for representing structured data based on JavaScript object syntax. It is commonly used for transmitting data in web applications (e.g., sending some data from the server to the client, so it can be displayed on a web page, or vice versa).

A JSON object is a structure consisting of key-value pairs separated by a colon as shown below:

```
{
  “Name”:”Hussam Mahfouz”,
  “Street Address”: “123 Secret Rd”,
  “Province”: “AB”,
  “Postal Code”: “A1B 2C3”,
  “Country”: Canada,
  }
```

In the above JSON object, the keys e.g. “Name”, “Street Address” e.t.c. exist as strings as do the values e.g. “Hussam Mahfouz”, “123 Secret Rd”, “AB”e.t.c.

This is not strictly necessary. In fact the values can be boolean, integers and doubles. In fact the values can even be nested objects or arrays or arrays of nested objects.

```
{
 "squadName": "Super hero squad",         // string value
 "homeTown": "Metro City",                // string value
 "formed": 2016,                          // integer value
 "secretBase": "Super tower",             // string value
 "active": true,                          // boolean value
 "members": [                             // array of objects
  {                                 // first object in “members” array
   "name": "Molecule Man",
   "age": 29,
   "secretIdentity": "Dan Jukes",
   "powers": [                      // array of strings within
    "Radiation resistance",           // first object in “members” array
    "Turning tiny",
    "Radiation blast"
   ]
  },
  {                                 // Second object in “members” array
   "name": "Madame Uppercut",
   "age": 39,
   "secretIdentity": "Jane Wilson",
   "powers": [
    "Million tonne punch",
    "Damage resistance",
```

```
      "Superhuman reflexes"
     ]
    },
    {                              // Third object in “members” array
     "name": "Eternal Flame",
     "age": 1000000,
     "secretIdentity": "Unknown",
     "powers": [
      "Immortality",
      "Heat Immunity",
      "Inferno",
      "Teleportation",
      "Interdimensional travel"
     ]
    }
  ]
}
```

Note how each object must exist within curly braces “{}”. Each array exists within square braces “[]”. JSON is such a a versatile format that some databases such as MongoDB organize the structure of all data stored around the JSON structure.

While one can think of JSON as a ‘structure`, it is typically ‘serialized’ into an array of bytes or array of characters before the sender transmits it. This is known as JSON serialization. On the receiving end, the receiver will receive this byte array and convert it into a JSON object that can then be easily interepreted. This is known as JSON deserialization.

In Qt, the following code snippet creates a Json object and then serializes it:

```
//create JSON object and fill in the key value pairs
QJsonObject objJson;
objJson.insert("Name","Hussam Mahfouz");
objJson.insert("Street Address","123 Secret Rd ");
objJson.insert("Province","AB");
objJson.insert("Postal Code","A1B 2C3");
objJson.insert("Country","Canada");
objJson.insert("Acknowledged", "??");
// convert the JSON object to an intermediary document
QJsonDocument docJson;
docJson.setObject(objJson);
// serialize the json document into a byte array that can be sent out using UDP
QByteArray byteArray = docJson.toJson(QJsonDocument::Compact);
```

A serialized JSON object looks like:

"{\"Acknowledged\":\"??\",\"Country\":\"Canada\",\"Name\":\"Hussam Mahfouz\",\"Postal Code\":\"A1B 2C3\",\"Province\":\"AB\",\"Street Address\":\"123 Secret Rd\"} "

To deserialize it back into a JSON Object:

```
//first convert the byte array into a JSON document and then to a JSON object
```

```
QJsonDocument document = QJsonDocument::fromJson(receivedByteArray);
QJsonObject jsonObj = document.object();
//Once the JSON object is created, the values in the JSON structure can be accessed via their keys using array (map) notation:
QString ackMsg = jsonObj["Name"].toString(); //ackMsg contains "Hussam Mahfouz"
```

On the Arduino side you will need to use the ArduinoJson library. To create and serialize a JSON object:

```
DynamicJsonDocument doc(1024);

doc["Name"] = "Hussam Mahfouz";
doc["Street Address"]   = "123 Secret Rd";
doc["Province"]   = "AB";
doc["Postal Code"]   = "A1B 2C3";
doc["Country"] = "Canada";


serializeJson(doc, Serial);
// This prints:
// "{\"Acknowledged\":\"??\",\"Country\":\"Canada\",\"Name\":\"Hussam Mahfouz\",\"Postal Code\":\"A1B 2C3\",\"Province\":\"AB\",\"Street Address\":\"123 Secret Rd\"} "
```

To Deserialize the same JSON object:

```
//packetBuffer is the byte array data captured from the UDP packets
DynamicJsonDocument docRx(512); //
DeserializationError error = deserializeJson(docRx, packetBuffer);

// Test if parsing succeeds.
if (error) {
  Serial.print(F("deserializeJson() failed: "));
  Serial.println(error.f_str());
  return;
}
//Now we can extract all the values in the JSON object
String namestr = docRx["Name"];
String staddrstr = docRx["Street Address"];
String provstr = docRx["Province"];
String poststr = docRx["Postal Code"];
String countrystr = docRx["Country"];
Serial.print("Name: "); Serial.println(namestr);
Serial.print("Street Address: "); Serial.println(staddrstr);
Serial.print("Province: "); Serial.println(provstr);
Serial.print("Postal Code: "); Serial.println(poststr);
Serial.print("Country: "); Serial.println(countrystr);
```

User Datagram Protocol (UDP)

UDP is a lossy network transmission protocol that transmits data between sender and receiver with no guarantees on the integrity of the data. UDP is ideal for transmitting small chunks of data such as JSON

objects. It's simple to implement and doesn't need to do all the various data integrity checks common in TCP. This means that it runs well on devices with limited computational power such as an Arduino Nano 33 IOT. It also results in some power savings.

In order to use the QUDPSocket library in Qt, the programmer is going to have to add the word 'network' to the first line of the .pro Qt project file:

```
QT      += core gui network
```

They will then need to include the following header files in the project header file:

```
#include <QtNetwork>
#include <QUdpSocket>
```

Then declare a pointer to a UDPSocket object in the project header file:

```
QUdpSocket *udpSocket;
```

In the constructor, allocate the QudpSocket object on the heap and bind it to a port. The QhostAddress::*AnyIPv4* flag allows the socket to connect to ports on other machines with different IP addresses. The QUdpSocket::*ShareAddress* flag allows the same UDP socket object to be used for both sending and receiving. The socket is bound to port 8877, so that the Qt Application will always be listening for UDP packet on port 8877. Finally the connect call ensures that whenever any data is received on the socket, the slot function is run to process this data

```
udpSocket = new QUdpSocket(this);
bool result = udpSocket->bind(QHostAddress::AnyIPv4,8877,QUdpSocket::ShareAddress);
qDebug() << result;
if(result){
            qDebug() << "PASS";
 }else{
            qDebug() << "FAIL";
 }
 connect(udpSocket, SIGNAL(readyRead()),this, SLOT(readUDPDatareceived()));
```

To send UDP data in byte array format to the Arduino Nano 33 IOT, use the following call:

```
udpSocket->writeDatagram(byteArray.data(),byteArray.size(),address, port);
```

To receive UDP data in byte array format from the Arduino Nano 33 IOT, use the following call:

```
QNetworkDatagram datagram = udpSocket->receiveDatagram();
```

This call is typically made inside the custom slot 'readUDPDatareceived()'.

On the Arduino Side after connecting to the WiFi network you'll need to make the following call to start the UDP server:

```
Udp.begin(localPort); // localPort is 2390
```

This starts the UDP server on port 2390 on the Arduino Nano 33 IOT, The in the loop function:

```
int packetSize = Udp.parsePacket();
int len = Udp.read(packetBuffer, 255);
```

The first line parses the incoming packet and the second line places the contents of the incoming packet into the packetBuffer array.

To send data in the 'ReplyBuffer' back over UDP to the Qt Application (listening on the PC on port 8877):

```
Udp.beginPacket(Udp.remoteIP(), 8877);
Udp.write(ReplyBuffer);
Udp.endPacket();
```

The complete Qt Application code is provided below. Remember that the Qt Application listens on port 8877, whereas the Arduino Nano 33 IOT listens on port 2390.

```
//widget.h
#ifndef WIDGET_H
#define WIDGET_H

#include <QWidget>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QLabel>
#include <QLineEdit>
#include <QPushButton>
#include <QComboBox>
#include <QValidator>
#include <QRegExpValidator>
#include <QJsonObject>
#include <QJsonDocument>
#include <QDebug>
#include <QtNetwork>
#include <QUdpSocket>

class Widget : public QWidget
{
   Q_OBJECT

public:
   Widget(QWidget *parent = 0);
   ~Widget();
public slots:
   void sendJsonUdp(void);
   void readUDPDatareceived(void);
private:
   QVBoxLayout *vbox, *vboxlbl, *vboxfield;
   QHBoxLayout *hbox;
   QLabel *namelbl, *staddrlbl, *provlbl, *postlbl, *countrylbl, *senderiplbl, *senderportlbl, *recvstatuslbl;
```

```
    QLineEdit *nameedt, *straddredt, *postedt, *countryedt, *senderipedt, *senderportedt;
    QPushButton *sendbtn;
    QComboBox *provbox;
    QRegExpValidator *postalcoderegexp;
    QUdpSocket *udpSocket;
    int okcount;
};

#endif // WIDGET_H

//widget.cpp
#include "widget.h"
Widget::Widget(QWidget *parent)
    : QWidget(parent)
{
    vboxlbl = new QVBoxLayout();
    vboxfield = new QVBoxLayout();
    vbox = new QVBoxLayout();
    hbox = new QHBoxLayout();

    namelbl =       new QLabel("Name:          ");
    staddrlbl =     new QLabel("Street Address: ");
    provlbl =       new QLabel("Province:       ");
    postlbl =       new QLabel("Postal Code:    ");
    countrylbl =    new QLabel("Country:        ");
    senderiplbl =   new QLabel("Sender IP:      ");
    senderportlbl = new QLabel("Sender Port:    ");
    recvstatuslbl = new QLabel("Received:       ");

    nameedt = new QLineEdit();
    straddredt = new QLineEdit();
    postedt = new QLineEdit();
    postalcoderegexp = new QRegExpValidator(QRegExp("[A-Za-z]{1}[0-9]{1}[A-Za-z]{1} [0-9]{1}[A-Za-z]{1}[0-9]{1}"));
    postedt->setValidator(postalcoderegexp);
    countryedt = new QLineEdit();
    countryedt->setReadOnly(true);countryedt->setText("Canada");
    senderipedt = new QLineEdit(); //senderipedt->setText("192.168.0.105");
    senderportedt = new QLineEdit();//senderportedt->setText("2390");

    sendbtn = new QPushButton("Send Address");
    provbox = new QComboBox();
    QStringList provStrList = {"AB", "BC", "MB", "NB", "NL", "NS","NT","NU", "ON", "PE","QC", "SK","YT"};
    provbox->addItems(provStrList);

   vboxlbl->addWidget(namelbl);
   vboxlbl->addWidget(staddrlbl);
   vboxlbl->addWidget(provlbl);
   vboxlbl->addWidget(postlbl);
   vboxlbl->addWidget(countrylbl);
   vboxlbl->addWidget(senderiplbl);
   vboxlbl->addWidget(senderportlbl);

   vboxfield->addWidget(nameedt);
   vboxfield->addWidget(straddredt);
```

```
    vboxfield->addWidget(provbox);
    vboxfield->addWidget(postedt);
    vboxfield->addWidget(countryedt);
    vboxfield->addWidget(senderipedt);
    vboxfield->addWidget(senderportedt);

    hbox->addLayout(vboxlbl);
    hbox->addLayout(vboxfield);
    vbox->addLayout(hbox);
    vbox->addWidget(sendbtn);
    vbox->addWidget(recvstatuslbl);

    setLayout(vbox);
    okcount = 0;
    udpSocket = new QUdpSocket(this);
    bool result = udpSocket->bind(QHostAddress::AnyIPv4, 8877,QUdpSocket::ShareAddress);
    qDebug() << result;
    if(result){
     qDebug() << "PASS";
    }else{
     qDebug() << "FAIL";
    }
    connect(udpSocket, SIGNAL(readyRead()),this, SLOT(readUDPDatareceived()));
    connect(sendbtn, SIGNAL(clicked()),this, SLOT(sendJsonUdp()));
}

Widget::~Widget(){
}

void Widget::sendJsonUdp(void){
    QJsonObject objJson;
    objJson.insert("Name",nameedt->text());
    objJson.insert("Street Address",straddredt->text());
    objJson.insert("Province",provbox->currentText());
    objJson.insert("Postal Code",postedt->text());
    objJson.insert("Country",countryedt->text());
    objJson.insert("Acknowledged", "??");
    QJsonDocument docJson;
    docJson.setObject(objJson);
    QByteArray byteArray = docJson.toJson(QJsonDocument::Compact);
    qDebug() << byteArray << endl;

    unsigned short port = senderportedt->text().toUShort();
    QHostAddress address;
    address.setAddress(senderipedt->text());
    udpSocket->writeDatagram(byteArray.data(),byteArray.size(),
                             address, port);

}

void Widget::readUDPDatareceived(void){
    qDebug() << "received datagram" << endl;
    while (udpSocket->hasPendingDatagrams()) {
        qDebug() << "found pending datagram" << endl;
```

```
        QNetworkDatagram datagram = udpSocket->receiveDatagram();
        qDebug() << datagram.data() << endl ;
        QJsonDocument document = QJsonDocument::fromJson(datagram.data());
        QJsonObject jsonObj = document.object();
        QString ackMsg = jsonObj["Acknowledged"].toString();
        qDebug() << ackMsg << endl;
        if(ackMsg == "OK") okcount++;

        recvstatuslbl->setText("Received: " + ackMsg + " " + QString::number(okcount));

    }
}

//main.cpp
#include "widget.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    Widget w;
    w.show();

    return a.exec();
}
```

Exercise 1. Create a new member function called drawApp() that instantiates all the widgets and puts them in their correct layout manager then sets the overall layout. Call this function in the constructor instead of just having all the code floating around in the constructor in addition to everything else.

(added drawapp() in header file class)

```
void Widget::drawApp() {
    vboxlbl = new QVBoxLayout();
    vboxfield = new QVBoxLayout();
    vbox = new QVBoxLayout();
    hbox = new QHBoxLayout();

    namelbl = new QLabel("Name:            ");
    staddrlbl = new QLabel("Street Address: ");
    provlbl = new QLabel("Province:        ");
    postlbl = new QLabel("Postal Code:     ");
    countrylbl = new QLabel("Country:         ");
    senderiplbl = new QLabel("Sender IP:       ");
    senderportlbl = new QLabel("Sender Port:     ");
    recvstatuslbl = new QLabel("Received:        ");

    nameedt = new QLineEdit();
    straddredt = new QLineEdit();
    postedt = new QLineEdit();

    postalcoderegexp = new QRegExpValidator(QRegExp("[A-Za-z]{1}[0-9]{1}[A-Za-z]{1}
[0-9]{1}[A-Za-z]{1}[0-9]{1}"));
    postedt->setValidator(postalcoderegexp);
    countryedt = new QLineEdit();
```

```
    countryedt->setReadOnly(true);
    countryedt->setText("Canada");
    senderipedt = new QLineEdit();
    senderportedt = new QLineEdit();
    sendbtn = new QPushButton("Send Address");
    provbox = new QComboBox();
    QStringList provStrList = {"AB", "BC", "MB", "NB", "NL", "NS", "NT", "NU",
"ON", "PE", "QC", "SK", "YT"};
    provbox->addItems(provStrList);

    vboxlbl->addWidget(namelbl);
    vboxlbl->addWidget(staddrlbl);
    vboxlbl->addWidget(provlbl);
    vboxlbl->addWidget(postlbl);
    vboxlbl->addWidget(countrylbl);
    vboxlbl->addWidget(senderiplbl);
    vboxlbl->addWidget(senderportlbl);

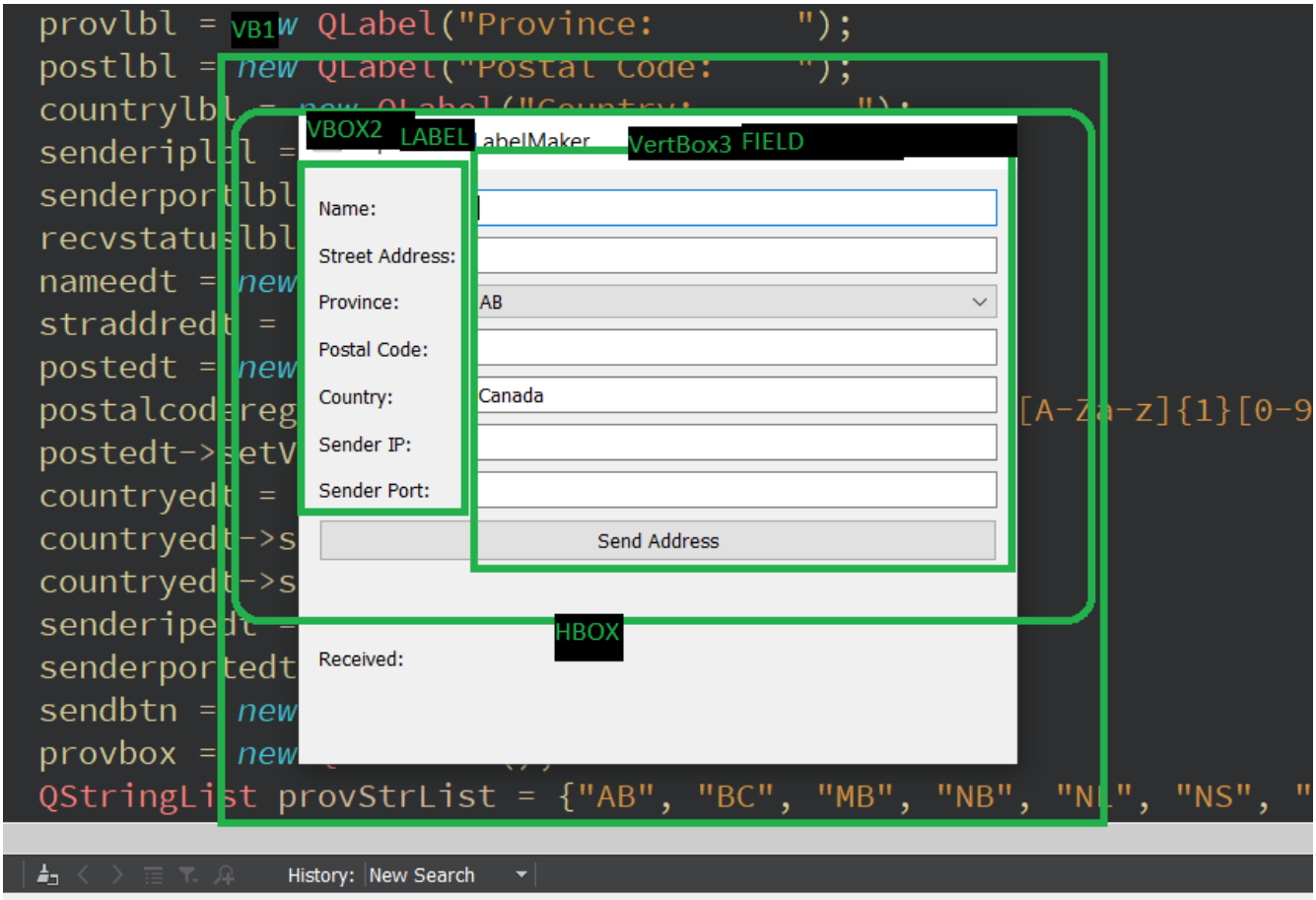
    vboxfield->addWidget(nameedt);
    vboxfield->addWidget(straddredt);
    vboxfield->addWidget(provbox);
    vboxfield->addWidget(postedt);
    vboxfield->addWidget(countryedt);
    vboxfield->addWidget(senderipedt);
    vboxfield->addWidget(senderportedt);

    hbox->addLayout(vboxlbl);
    hbox->addLayout(vboxfield);

    vbox->addLayout(hbox);
    vbox->addWidget(sendbtn);
    vbox->addWidget(recvstatuslbl);

    setLayout(vbox);
}
```

Exercise 2. Trace out the layout managers vbox, vboxlbl, vboxfield & hbox on the image of the application show in figure 1. You can always do this digitally in paintbrush. Basically idemtify the location and contents of each layout manager.



Exercise 3. In your own words describe the purpose of the 'postalcoderegexp' object.
Specifies input format for the postal code. The user can only enter the format (xxx xxx) ie : (J9J 0M5).

Exercise 4. In your own words describe the purpose of the 'okcount' variable:
Counts number of "ok" strings (acknowledgments) received in the json object.

Exercise 5. In your own words describe the purpose of this line of code:

```
countryedt->setReadOnly(true);countryedt->setText("Canada")
```

;
Sets the input field for country to read only meaning the user cant write in it. Its value get prefilled with the text 'Canada'.

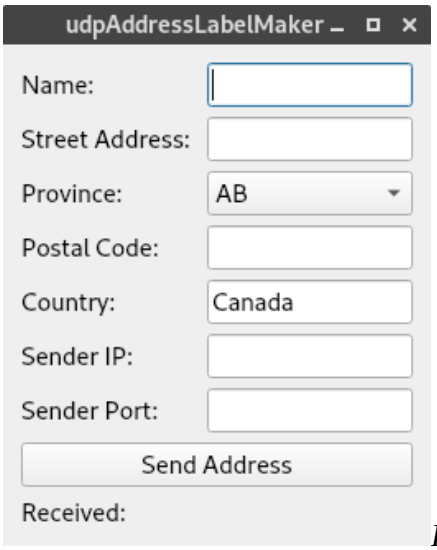


Figure 1

The Arduino code is provided below:

```
#include <ArduinoJson.h>
#include <SPI.h>
#include <WiFiNINA.h>
#include <WiFiUdp.h>
#include "arduino_secrets.h"

void printWifiStatus(void);
void connectToWifi(void);

int status = WL_IDLE_STATUS;
///////please enter your sensitive data in the Secret tab/arduino_secrets.h
char ssid[] = SECRET_SSID;        // your network SSID (name)
char pass[] = SECRET_PASS;    // your network password (use for WPA, or use as key for WEP)
int keyIndex = 0;            // your network key index number (needed only for WEP)

unsigned int localPort = 2390;      // local port to listen on

char packetBuffer[512]; //buffer to hold incoming packet
char  ReplyBuffer[512];       // a string to send back

WiFiUDP Udp;

void setup() {
  //Initialize serial and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for native USB port only
  }

  connectToWifi();
  Serial.println("\nStarting connection to server...");
  // if you get a connection, report back via serial:
  Udp.begin(localPort);
```

```
}

void loop() {
  // if there's data available, read a packet
  int packetSize = Udp.parsePacket();
  if (packetSize) {
    Serial.print("Received packet of size ");
    Serial.println(packetSize);
    Serial.print("From ");
    IPAddress remoteIp = Udp.remoteIP();
    Serial.print(remoteIp);
    Serial.print(", port ");
    Serial.println(Udp.remotePort());

    // read the packet into packetBufffer
    int len = Udp.read(packetBuffer, 255);
    if (len > 0) {
      packetBuffer[len] = 0;
    }
    Serial.println("Contents:");
    Serial.println(packetBuffer);
    DynamicJsonDocument docRx(512);
    DeserializationError error = deserializeJson(docRx, packetBuffer);

    // Test if parsing succeeds.
    if (error) {
      Serial.print(F("deserializeJson() failed: "));
      Serial.println(error.f_str());
      return;
    }
    String namestr = docRx["Name"];
    String staddrstr = docRx["Street Address"];
    String provstr = docRx["Province"];
    String poststr = docRx["Postal Code"];
    String countrystr = docRx["Country"];
    Serial.print("Name: "); Serial.println(namestr);
    Serial.print("Street Address: "); Serial.println(staddrstr);
    Serial.print("Province: "); Serial.println(provstr);
    Serial.print("Postal Code: "); Serial.println(poststr);
    Serial.print("Country: "); Serial.println(countrystr);

    docRx["Acknowledged"] = "OK";
    serializeJson(docRx, ReplyBuffer);
    serializeJson(docRx, Serial);
    // send a reply, to the IP address and port that sent us the packet we received
    Udp.beginPacket(Udp.remoteIP(), 8877);
    Udp.write(ReplyBuffer);
    Udp.endPacket();
    delay(1000);
  }
}

void connectToWifi(void) {
    // check for the WiFi module:
    if (WiFi.status() == WL_NO_MODULE) {
```

```
    Serial.println("Communication with WiFi module failed!");
    // don't continue
    while (true);
  }

  String fv = WiFi.firmwareVersion();
  if (fv < WIFI_FIRMWARE_LATEST_VERSION) {
    Serial.println("Please upgrade the firmware");
  }

  // attempt to connect to WiFi network:
  while (status != WL_CONNECTED) {
    Serial.print("Attempting to connect to SSID: ");
    Serial.println(ssid);
    // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
    status = WiFi.begin(ssid, pass);

    // wait 10 seconds for connection:
    delay(10000);
  }
  Serial.println("Connected to WiFi");
  printWifiStatus();

}


void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your board's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print("signal strength (RSSI):");
  Serial.print(rssi);
  Serial.println(" dBm");
}
```

In the Arduino Editor, create a new tab/header file called ‘arduino_secrets.h’and type:

```
#define SECRET_SSID "YOURSSID"
#define SECRET_PASS "YOURWIFIPASSWORD"
```

Exercise 6. Compile the download the Arduino code provided above on the Arduino Nano 33 IOT and open the serial terminal. Wait until the Arduino connects to the network. Then Compile and run the Qt Code. Type in some address data, the IP address of Arduino (should be available in Arduino serial terminal), port that the Arduino is listening on...2390 and press the ‘send Address’ button. Pay close attention to both the serial terminal in the arduino, the application output window in QtCreator and the Qt application itself. In your own words describe what is happening.

QT program: The data the user inputs in the text fields is sent to the specified IP address and port after the send button is pressed. This serializes the user data into a JSON object and sends it over UDP to the Arduino(via the IP/Port).

Arduino: Connects to network using WifiNina library then acts as a UDP server. The Arduino listens to port 2390 for packets. The code receives a UDP packet, processes it, and deserializes the data from the JSON object and throws the data into strings. The nano then sends an acknowledgment showing the received address data with an ‘ok’ in the qt app over UDP. The Arduino serial terminal also displays the received address, and the Qt application's acknowledgment is shown there too.

Exercise 7. In your own words explain what the function ‘connectToWifi()’ (Arduino Code) does line by line.

void connectToWifi(void) {: //beginning of function

if (WiFi.status() == WL_NO_MODULE) {: //Checks to see if there’s an onboard wifi adapter.

Serial.println("Communication with WiFi module failed!");: //prints an error message if theres no wifi adapter.

while (true);: //loop for when theres no wifi adapter

String fv = WiFi.firmwareVersion();: //finds out firmware version of wifi and puts into a string

if (fv < WIFI_FIRMWARE_LATEST_VERSION) {: //checks if current firmware is less than latest firmware.
Serial.println("Please upgrade the firmware");: prints line in monitor if firmware too old

---

while (status != WL_CONNECTED) {: creates a loop for when the wifi is connected

Serial.print("Attempting to connect to SSID: ");: prints attempt to serial monitor

Serial.println(ssid);: This line prints the ssid.

status = WiFi.begin(ssid, pass);: attempts to connect to the network using the variables from the secret.ino file

delay(10000);: 10sec delay after trying to connect

Serial.println("Connected to WiFi");: prints when Arduino connected to network

printWifiStatus();: prints additional network info like signal strength.

Exercise 8. Add an additional key-value pair:
“Quote”: “Your favourite Quote”
to the JSON structure. Modify both the Qt application code and the Arduino code to have them both bounce around this additional piece of information between themselves.

QT:

```
void Widget::sendJsonUdp(void)
{
    QJsonObject objJson;
    objJson.insert("Name", nameedt->text());
    objJson.insert("Street Address", straddredt->text());
    objJson.insert("Province", provbox->currentText());
    objJson.insert("Postal Code", postedt->text());
    objJson.insert("Country", countryedt->text());
    objJson.insert("Quote", "IT WAS A MISINPUT !!!!"); //added quote
```

Arduino Changes:

….

```
  String namestr = docRx["Name"];
   String staddrstr = docRx["Street Address"];
   String provstr = docRx["Province"];
   String poststr = docRx["Postal Code"];
   String countrystr = docRx["Country"];
   String quotestr = docRx["Quote"]; // QUOTE FIELD

   Serial.print("Name: ");
   Serial.println(namestr);
   Serial.print("Street Address: ");
   Serial.println(staddrstr);
   Serial.print("Province: ");
   Serial.println(provstr);
   Serial.print("Postal Code: ");
   Serial.println(poststr);
   Serial.print("Country: ");
   Serial.println(countrystr);
   Serial.print("Quote: ");
   Serial.println(quotestr); // PRINT QUOTE STUFF
```

…

```
23:42:33: Starting C:\Users\jamie\Desktop\QtProj\build-udpAddressLabelMaker-Desktop_Qt_5_12_12_MinGW_64_bit-Debug\debug\udpAddressLabelMaker.exe
true
PASS
"{\"Country\":\"Canada\",\"Name\":\"\",\"Postal Code\":\"\",\"Province\":\"AB\",\"Quote\":\"IT WAS A MISINPUT!!!!\",\"Street Address\":\"\"}"
```

Activate Windows
Go to Settings to activate